CHAT GPT

Problema: Olvidar tareas o compromisos por falta de organización

2. Estructura recomendada: Cola de prioridad

La **cola de prioridad** es adecuada porque permite organizar las tareas según su nivel de **urgencia o importancia**. Las tareas más importantes o próximas para vencer se atienden primero.

Ejemplo:

- Entregar trabajo mañana: Alta prioridad
- Estudiar para un examen: Media prioridad
- Ordenar el escritorio: Baja prioridad

Ventajas:

- Permite organizar las tareas por importancia.
- Ayuda a atender primero los compromisos urgentes.
- Facilita agregar nuevas tareas.
- Reduce la posibilidad de olvidar tareas importantes.

Desventajas:

- Una tarea de baja prioridad podría quedar olvidada.
- Hay que asignar correctamente la prioridad.
- Si varias tareas tienen la misma prioridad, se necesita otro criterio, como la fecha límite.

3. Alternativa: Lista enlazada

Una **lista enlazada** también podría utilizarse para almacenar las tareas, permitiendo agregar, eliminar y recorrer compromisos fácilmente.

Ventajas:

- Es flexible para agregar y eliminar tareas.
- No necesita tener un tamaño fijo.
- Es sencilla de recorrer y modificar.

Desventajas:

- No organiza automáticamente las tareas por prioridad.
- Buscar una tarea específica puede ser más lento.
- Para saber cuál tarea es más urgente habría que recorrer o mantener ordenada la lista.

4. Contraste con lo visto en clase

La **cola de prioridad** sería más adecuada si en clase se ha trabajado con **colas, prioridades o estructuras que permiten organizar elementos según un criterio**. La lista enlazada sería más general, pero requeriría implementar manualmente la organización por prioridad.

5. Aciertos, vacíos y posibles errores

Aciertos:
La cola de prioridad permite atender primero las tareas más urgentes y ayuda a organizar los compromisos.

Vacíos:
No considera por sí sola otros factores como duración de la tarea, fecha exacta o cantidad de tiempo disponible.

Posible error:
No todas las tareas deben clasificarse únicamente por importancia. Una tarea poco importante, pero con una fecha límite cercana también podría necesitar una prioridad alta.

Conclusión: Para este problema, elegiría la **cola de prioridad**, porque el objetivo principal es evitar olvidar compromisos y permite identificar rápidamente qué tarea debe realizarse primero.

Claude

Estructura recomendada por la IA

Cola de prioridad (Priority Queue) con Heap

- Permite obtener siempre la tarea más urgente en O(1) e insertar en O(log n).
- Ideal porque las tareas tienen distinta urgencia (no todas pesan igual).

- Desventajas: más compleja de implementar, no preserva el orden de llegada entre tareas iguales, reordenar prioridades es costoso, y es "excesiva" si hay pocas tareas.

Alternativa y comparación

Lista enlazada ordenada

- Ventaja principal: siempre está ordenada, fácil de recorrer completa y más simple de programar.
- Desventaja principal: insertar una nueva tarea es lento (O(n)) porque hay que recorrer la lista para encontrar su lugar.

Contraste con lo discutido en clase

(Esto depende de lo que se haya visto en tu curso; algunas guías típicas:)

- Si en clase se enfatizó **eficiencia y escalabilidad**, el heap es la elección que coincide con la teoría.
- Si en clase se enfatizó **simplicidad para principiantes** o estructuras básicas (arreglos, listas), la lista enlazada ordenada se ajusta mejor al nivel introductorio.
- Verifica si el profesor pidió específicamente estructuras como **árboles binarios de búsqueda (BST)** o **tablas hash**, ya que la IA no las consideró y podrían ser relevantes según el temario.

Aciertos, vacíos y posibles errores

Aciertos:

- La IA identificó correctamente que la urgencia/prioridad es el criterio clave del problema.
- Comparó complejidades (Big-O) de forma correcta.

Vacíos o posibles errores:

- No consideró estructuras híbridas (ej. **tabla hash + heap** para búsqueda rápida por nombre de tarea además de prioridad).
- No mencionó cómo manejar **tareas recurrentes** o **recordatorios con notificaciones**, que son parte real del problema de "organización".

- Asumió que la fecha límite es el único criterio de prioridad, sin considerar categorías (trabajo, personal, urgente/importante — matriz de Eisenhower).
- No discutió persistencia de datos (¿cómo se guardan las tareas si se cierra la app?).

DeepSeek

Estructura recomendada (IA):
Cola de prioridad (heap binario).

- **Por qué:** Acceso O (1) a la tarea más urgente (raíz); inserciones/eliminaciones O (log n). Ideal para recordatorios activos.
- **Desventajas:** Búsqueda arbitraria O(n); actualizar prioridad es costoso; no muestra lista ordenada completa sin destruirla; no soporta consultas por rango.

3. Alternativa (árbol AVL/Red-Black ordenado por fecha):

- **Ventajas:** Búsqueda/inserción/eliminación O (log n); recorrido in-order da vista global; consultas por rango eficientes.
- **Desventajas:** Implementación compleja; obtener la más urgente es O (log n) (no O (1)); mayor overhead de memoria.

4. Contraste con clase:

- En clase se prioriza la operación dominante. Aquí hay dos: urgencia (heap gana) y visión global (árbol gana).
- El heap resuelve el *recordatorio*, pero el árbol resuelve mejor la *organización* preventiva. Coincidimos en que no hay estructura perfecta sin conocer el uso real.

5. Aciertos, vacíos y errores de mi recomendación:

- Acierto: Señalé correctamente la eficiencia del heap para urgencia y su simplicidad.
- Vacío: No consideré categorías (trabajo/personal) ni tareas sin fecha fija.
- Error: Ignoré soluciones híbridas (heap + diccionario para actualizaciones) y la necesidad de persistencia en BD.
- Advertencia faltante: La función de comparación (fecha + importancia) debe definirse bien, o el heap falla.

Conclusión: Para este problema, el **árbol balanceado** es más completo por la visión global, pero el **heap** sirve como motor interno de notificaciones. En la práctica, se necesita una base de datos con índices.